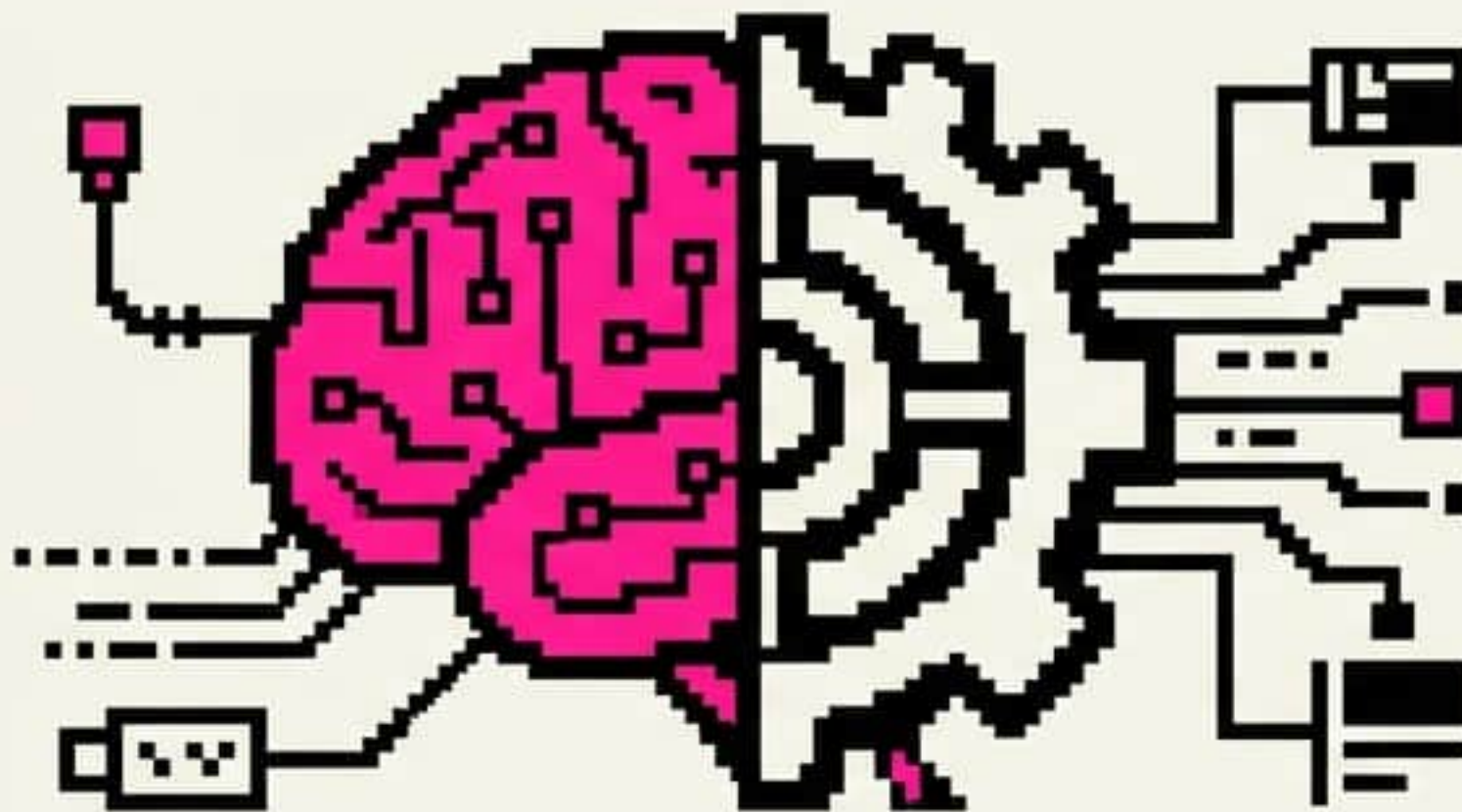


SỔ TAY VẬN HÀNH AI ORCHESTRATOR

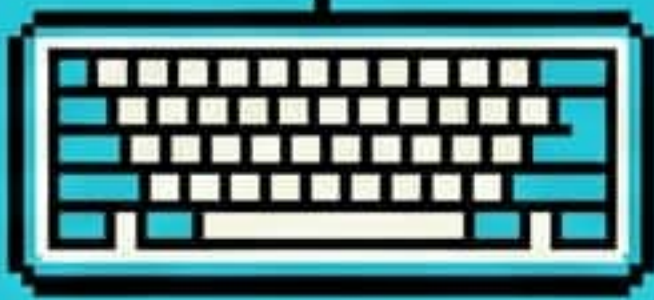
System Manual / Vol 1.0 / AI-Assisted Development



Tài liệu hướng dẫn chuyển đổi từ thợ viết code (syntax)
sang kỹ sư điều phối hệ thống (orchestration).

SỰ DỊCH CHUYỂN VAI TRÒ

The Operator / Mindset Shift



Thợ Viết Code (Old Way)

Gõ từng dòng lệnh (Syntax typing)

Tự tìm bug, tự debug chậm chạp



AI Orchestrator (New Way)

Điều phối hệ thống (System orchestration)

- 🎯 Định hướng: Xác định mục tiêu rõ ràng.
- 📋 Cung cấp Context: Cho AI đủ thông tin.
- ✅ Validate: Đánh giá code AI viết.
- ⚙️ Quyết định: Chọn architecture / approach.
- 🔄 Iterate: Feedback loop liên tục.

KIẾN TRÚC HỆ THỐNG AI

System Architecture / Stack Anatomy



LLM (Bộ Não)

Dự đoán token tiếp theo. KHÔNG có memory giữa các sessions.



Context Window (Bộ Nhớ Ngắn Hạn)

Bàn làm việc giới hạn. Bàn càng bừa, AI càng khó tập trung.



Tools (Bàn Tay)

Biến text thành hành động (đọc file, chạy terminal, mở web).



Tokens (Tiền Tệ)

Chi phí vận hành (Input + Output). Quản lý context = Tiết kiệm token.

LỰA CHỌN LỖI ĐỘNG CƠ

Core Engine Matrix / LLM Comparison

Claude Sonnet 4.7 / Claude Opus 4.7

Điểm mạnh: Reasoning sâu, tuân thủ instruction cực tốt, code dài hơi.

Điểm yếu: Chi phí cao.

GPT-5.3 Codex / GPT-5.4

Điểm mạnh: Tốc độ cực nhanh, Multi-modal tốt.

Điểm yếu: Đôi khi tự sáng tạo quá đà (hallucination).

Kimi 2.5

Điểm mạnh: Giá rẻ, khả năng code tương đồng Claude 4.5.

Deepseek

Điểm mạnh: Giá cực rẻ, khả năng coding xuất sắc.

Điểm yếu: Tốc độ chậm, hỗ trợ ít tools hơn.

QUYỀN NĂNG THỰC THI

Tool Arsenal / Execution Capabilities



File Read

Đọc source code, config, docs.



File Write

Viết code, sửa bug, thêm tests.



Terminal

Chạy lệnh trực tiếp (npm test, git status).



Search

Quét codebase tìm pattern, reference.



Browser

Test UI, đọc docs online, chụp màn hình.



MCP

Kết nối external services, Database, APIs.

AI không chỉ nói. Nó hành động. (Agents execute tasks autonomously)

BẢN ĐỒ CÔNG CỤ 2025

-!- Landscape Matrix / IDE vs CLI

IDE-Based (e.g., Cursor, Windsurf)

- **Ưu điểm:** Giao diện trực quan, dễ onboard, inline suggestions.
- **Nhược điểm:** Ít kiểm soát hệ thống, chi phí cao.
- **Sử dụng:** Daily coding, UI/Frontend.
- **Đối tượng:** Juniors (Bắt đầu).

VS

CLI-Based (e.g., Claude Code)

- **Ưu điểm:** Sức mạnh tuyệt đối, automation, hỗ trợ subagent.
- **Nhược điểm:** Learning curve dốc, chỉ dùng terminal.
- **Sử dụng:** Complex tasks, architecture, debugging sâu.
- **Đối tượng:** Tất cả (Mục tiêu long-term).

PHÂN CẤP CONTEXT

Context Hierarchy / Resolution Order

Global Scope

Project Scope

Directory Scope

File: AGENTS.md (trong sub-folder).
Áp dụng: Cực kỳ specific cho từng module (VD: tests/, api/).

File: .cursorrules, CLAUDE.md (root).
Áp dụng: Rule chung cho toàn bộ repository.

File: ~/.claude/CLAUDE.md
Áp dụng: Tất cả projects. Load tự động.

Level sâu hơn thẳng

BẢN THIẾT KẾ CẤU HÌNH

Configuration Anatomy / CLAUDE.md Blueprint

CLAUDE.md

[Directory Structure]: Chỉ định rõ nơi chứa models, schemas, services, tests.

[Database Patterns]: Ép AI dùng SQLAlchemy 2.0, Alembic migrations.

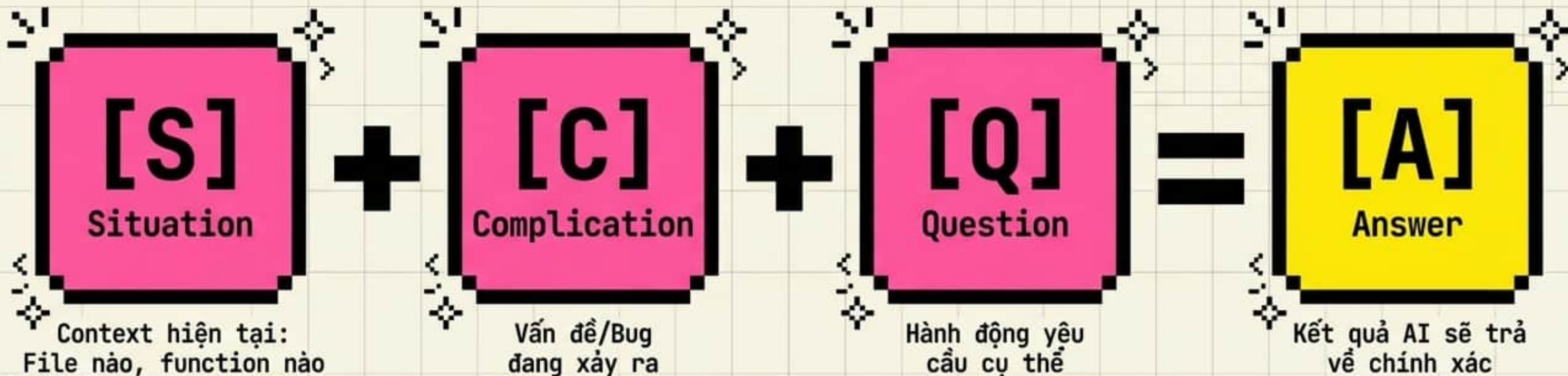
[Known Gotchas]: (Quan trọng nhất) Cảnh báo AI: DB sessions auto-close, Pydantic v2 dùng model_validate, Async routes cần await.

[DO NOT]: Lệnh cấm tuyệt đối: Không mock DB trong API tests, Không share state giữa các tests.

File cấu hình chính là '**Bản mô tả công việc**' bắt buộc AI phải tuân thủ.

CÔNG THỨC SCQA

Prompt Framework / Precision Engineering



✗ Prompt Kém

"Fix bug login cho tôi" -> AI phải tốn token quét toàn bộ codebase, dễ sai.

✓ Prompt Tốt

"Trong auth.py, hàm login(), token hết hạn đang crash app. Hãy thêm try/catch trả về 401."
-> Zero token lãng phí, chính xác tuyệt đối.

GIAO THỨC THỰC THI

Execution Protocols / Rules of Engagement

1 | READ BEFORE CODE: Yêu cầu AI đọc context file trước khi viết dòng code đầu tiên.

[]

2 | FOLLOW PATTERNS: Ép AI tuân thủ existing codebase patterns, không tự chế "bánh xe".

3 | TEST BEFORE CODE: Viết test file trước khi implement (TDD skill).

4 | ATOMIC REQUESTS: 1 Task = 1 Request = 1 Commit. Không gộp việc.

5 | VERIFY BEFORE ACCEPT: Validate thủ công trước khi merge code AI tạo ra.

MA TRẬN ĐÁNH GIÁ RỦI RO

Trust vs. Verify / Risk Matrix

TIN TƯỜNG (Low Risk)

- Code theo pattern có sẵn / Boilerplate.
- Documentation / Code comments.
- Code formatting / Refactoring đơn giản.
- Simple CRUD operations.

Lý do: Cấu trúc rõ ràng, test dễ catch lỗi.

KIỂM CHỨNG KỸ (High Risk)

- Security, Auth, Encryption.
- Complex DB Queries & Transactions.
- Business logic (Tính toán tiền, Rules).
- Concurrency, Async code, Race conditions.

Lý do: AI không hiểu business context, xử lý edge case kém.

QUẢN TRỊ BỘ NHỚ

Session Management / Context Hygiene /

Trắng / Sạch

Đang nạp Files

Đầy / Ảo giác

Bàn làm việc bừa bộn

Mỗi session bắt đầu trắng. Càng **add nhiều files & chat dài** -> **Context window bị lấp đầy** -> AI mất tập trung, sinh **ảo giác (hallucinate)**.

KHI TASK HOÀN THÀNH -> CLEAR SESSION.
KHI CHUYỂN MODULE -> START NEW SESSION.
Đừng giữ 1 đoạn chat dài cho nhiều task khác nhau!

CHECKLIST CỦA ORCHESTRATOR

Pre-flight Synthesis / Final Protocol



Cung cấp file path + function name cụ thể (SCQA).



Yêu cầu AI đọc codebase/context files TRƯỚC.



Chỉ thị tuân thủ existing patterns (Theo CLAUDE.md).



Yêu cầu AI viết test trước khi viết code (TDD).



Chạy test cục bộ sau mỗi thay đổi của AI.



Đối chiếu rủi ro (Trust vs. Verify Matrix) trước khi accept.



Commit atomic (1 change = 1 commit).

Đừng gõ lại cú pháp. Hãy điều phối hệ thống.
(Stop typing syntax. Start orchestrating systems.)